

而枚举子集和枚举整数一样简单:

```
for(int i = 0; i < (1<<n); i++)    //枚举各子集所对应的编码 0, 1, 2, ..., 2^n-1
    print_subset(n, i);
```

提示 7-8: 从代码量看, 枚举子集的最简单方法是二进制法。

7.4 回溯法

无论是排列生成还是子集枚举, 前面都给出了两种思路: 递归构造和直接枚举。直接枚举法的优点是思路和程序都很简单, 缺点在于无法简便地减小枚举量——必须生成 (generate) 所有可能的解, 然后一一检查 (test)。

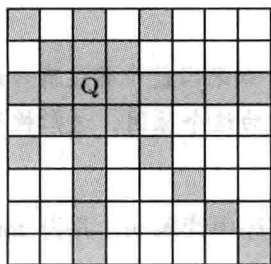
另一方面, 在递归构造中, 生成和检查过程可以有机结合起来, 从而减少不必要的枚举。这就是本节的主题——回溯法 (backtracking)。

回溯法的应用范围很广, 只要能把待求解的问题分成不太多的步骤, 每个步骤又只有不太多的选择, 都可以考虑应用回溯法。为什么说“不太多”呢? 想象一棵包含 L 层, 每层的分支因子均为 b 的解答树, 其结点数高达 $1+b+b^2+\dots+b^{L-1}=\frac{b^L-1}{b-1}$ 。无论是 b 太大还是 L 太大, 结点数都会是一个天文数字。

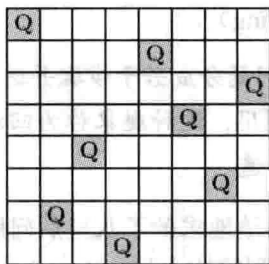
回溯法是初学者学习暴力法的第一个障碍, 学习时间短则数天, 长则数月甚至一年以上。为了减少不必要的困扰, 在学习回溯法之前, 请读者确保 7.2 节和 7.3 节的所有递归程序都可以熟练、准确地写出。

7.4.1 八皇后问题

在棋盘上放置 8 个皇后, 使得它们互不攻击, 此时每个皇后的攻击范围为同行同列和同对角线, 要求找出所有解, 如图 7-4 所示。



(a) 皇后的攻击范围



(b) 一个可行解

图 7-4 八皇后问题

【分析】

最简单的思路是把问题转化为“从 64 个格子中选一个子集”, 使得“子集中恰好有 8

个格子，且任意两个选出的格子都不在同一行、同一列或同一个对角线上”。这正是子集枚举问题。然而，64 个格子的子集有 2^{64} 个，太大了，这并不是一个很好的模型。

第二个思路是把问题转化为“从 64 个格子中选 8 个格子”，这是组合生成问题。根据组合数学，有 $C_{64}^8 = 4.426 \times 10^9$ 种方案，比第一种方案优秀，但仍然不够好。

经过思考，不难发现以下事实：恰好每行每列各放置一个皇后。如果用 $C[x]$ 表示第 x 行皇后的列编号，则问题变成了全排列生成问题。而 0~7 的排列一共只有 $8! = 40320$ 个，枚举量不会超过它。

提示 7-9：在编写递归枚举程序之前，需要深入分析问题，对模型精雕细琢。一般还应应对解答树的结点数有一个粗略的估计，作为评价模型的重要依据，如图 7-5 所示。

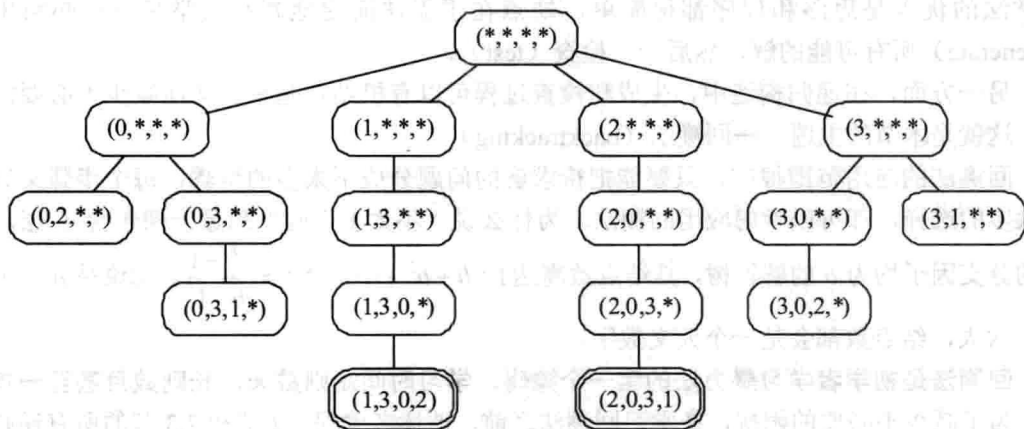


图 7-5 四皇后问题的解答树

图 7-5 中给出了四皇后问题的完整解答树。它只有 17 个结点，比 $4! = 24$ 小。为什么会这样呢？这是因为有些结点无法继续扩展。例如，在 $(0,2,*,*)$ 中，第 2 行无论将皇后放到哪里，都会和第 0 行和第 1 行中已放好的皇后发生冲突，其他还未放置的皇后更是如此。

在这种情况下，递归函数将不再递归调用它自身，而是返回上一层调用，这种现象称为回溯（backtracking）。

提示 7-10：当把问题分成若干步骤并递归求解时，如果当前步骤没有合法选择，则函数将返回上一级递归调用，这种现象称为回溯。正是因为这个原因，递归枚举算法常被称为回溯法，应用十分普遍。

下面的程序简洁地求解了八皇后问题。在主程序中读入 n ，并为 tot 清零，然后调用 search(0)，即可得到解的个数 tot。

```

void search(int cur) {
    if (cur == n) tot++; //递归边界。只要走到了这里，所有皇后必然不冲突
    else for (int i = 0; i < n; i++) {
        int ok = 1;

```

```

C[cur] = i; //尝试把第 cur 行的皇后放在第 i 列
for(int j = 0; j < cur; j++) //检查是否和前面的皇后冲突
    if(C[cur] == C[j] || cur-C[cur] == j-C[j] || cur+C[cur] == j+C[j])
        { ok = 0; break; }
if(ok) search(cur+1); //如果合法，则继续递归
}
}

```

注意：既然是逐行放置的，则皇后肯定不会横向攻击，因此只需检查是否纵向和斜向攻击即可。条件“ $cur-C[cur] == j-C[j] \parallel cur+C[cur] == j+C[j]$ ”用来判断皇后 $(cur, C[cur])$ 和 $(j, C[j])$ 是否在同一条对角线上。其原理可以用图 7-6 来说明。

0	1	2	3	4	5	6	7
-1	0	1	2	3	4	5	6
-2	-1	0	1	2	3	4	5
-3	-2	-1	0	1	2	3	4
-4	-3	-2	-1	0	1	2	3
-5	-4	-3	-2	-1	0	1	2
-6	-5	-4	-3	-2	-1	0	1
-7	-6	-5	-4	-3	-2	-1	0

(a) 格子 (x,y) 的 $y-x$ 值标识了主对角线

0	1	2	3	4	5	6	7
1	2	3	4	5	6	7	8
2	3	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	12
6	7	8	9	10	11	12	13
7	8	9	10	11	12	13	14

(b) 格子 (x,y) 的 $x+y$ 值标识了副对角线

图 7-6 棋盘中的对角线标识

结点数似乎很难进一步减少了，但程序效率可以继续提高：利用二维数组 `vis[2][]` 直接判断当前尝试的皇后所在的列和两个对角线是否已有其他皇后。注意到主对角线标识 $y-x$ 可能为负，存取时要加上 n 。

```

void search(int cur) {
    if(cur == n) tot++;
    else for(int i = 0; i < n; i++) {
        if(!vis[0][i] && !vis[1][cur+i] && !vis[2][cur-i+n]) {
            //利用二维数组直接判断
            C[cur] = i; //如果不用打印解，整个 C 数组都可以省略
            vis[0][i] = vis[1][cur+i] = vis[2][cur-i+n] = 1; //修改全局变量
            search(cur+1);
            vis[0][i] = vis[1][cur+i] = vis[2][cur-i+n] = 0; //切记！一定要改回来
        }
    }
}

```

上面的程序有个极其关键的地方：`vis` 数组的使用。`vis` 数组的确切含义是什么？它表示已经放置的皇后占据了哪些列、主对角线和副对角线。将来放置的皇后不应该修改这些值——至少“看上去没有修改”。一般地，如果在回溯法中修改了辅助的全局变量，则一

定要及时把它们恢复原状(除非故意保留所做修改)。若不信,可以把“`vis[0][i]=vis[1][cur+i]=vis[2][cur-i+n]=0`”注释掉,验证还能否正确求解八皇后问题。另外,在调用之前一定要把 `vis` 数组清空。

提示 7-11: 如果在回溯法中使用了辅助的全局变量,则一定要及时把它们恢复原状。特别地,若函数有多个出口,则需在每个出口处恢复被修改的值。

7.4.2 其他应用举例

例题 7-4 素数环 (Prime Ring Problem, UVa 524)

输入正整数 n , 把整数 $1, 2, 3, \dots, n$ 组成一个环, 使得相邻两个整数之和均为素数。输出时从整数 1 开始逆时针排列。同一个环应恰好输出一次。 $n \leq 16$ 。

样例输入:

6

样例输出:

1 4 3 2 5 6

1 6 5 2 3 4

【分析】

由模型不难得到: 每个环对应于 $1 \sim n$ 的一个排列, 但排列总数高达 $16! = 2 \times 10^{13}$, 生成测试法会超时吗? 下面进行实验:

```
for(int i = 2; i <= n*2; i++) isp[i] = is_prime(i); //生成素数表, 加快后续判断
for(int i = 0; i < n; i++) A[i] = i+1; //第一个排列
do {
    int ok = 1;
    for(int i = 0; i < n; i++) if(!isp[A[i]+A[(i+1)%n]]) { ok = 0; break; }
    //判断合法性
    if(ok){
        for(int i = 0; i < n; i++) printf("%d ", A[i]); //输出序列
        printf("\n");
    }
}while(next_permutation(A+1, A+n)); //1 的位置不变
```

运行后发现, 当 $n=12$ 时就已经很慢, 而当 $n=16$ 时无法运行出结果。下面试试回溯法:

```
void dfs(int cur){
    if(cur == n && isp[A[0]+A[n-1]]){ //递归边界。别忘了测试第一个数和最后一个数
        for(int i = 0; i < n; i++) printf("%d ", A[i]); //打印方案
        printf("\n");
    }
}
```

```

else for(int i = 2; i <= n; i++) //尝试放置每个数 i
    if(!vis[i] && isp[i+A[cur-1]]){ //如果 i 没有用过, 并且与前一个数之和为素数
        A[cur] = i;
        vis[i] = 1; //设置使用标志
        dfs(cur+1);
        vis[i] = 0; //清除标志
    }
}

```

回溯法比生成-测试法快了很多, 即使 $n=18$ 速度也不错。将上面的函数名设为 `dfs` 并不是巧合——从解答树的角度讲, 回溯法正是按照深度优先的顺序在遍历解答树。在后面的内容中, 还将学习更多遍历解答树的方法。

提示 7-12: 如果最坏情况下的枚举量很大, 应该使用回溯法而不是生成-测试法。

例题 7-5 困难的串 (Krypton Factor, UVa 129)

如果一个字符串包含两个相邻的重复子串, 则称它是“容易的串”, 其他串称为“困难的串”。例如, BB、ABCDACABCAB、ABCDABCD 都是容易的串, 而 D、DC、ABDAB、CBABCBA 都是困难的串。

输入正整数 n 和 L , 输出由前 L 个字符组成的、字典序第 k 小的困难的串。例如, 当 $L=3$ 时, 前 7 个困难的串分别为 A、AB、ABA、ABAC、ABACA、ABACAB、ABACABA。输入保证答案不超过 80 个字符。

样例输入:

```

7 3
30 3

```

样例输出:

```

ABACABA
ABACABCACBABCABACABCACBACABA

```

【分析】

基本框架不难确定: 从左到右依次考虑每个位置上的字符。因此, 问题的关键在于如何判断当前字符串是否已经存在连续的重复子串。例如, 如何判断 ABACABA 是否包含连续重复子串呢? 一种方法是检查所有长度为偶数的子串, 分别判断每个子串的前一半是否等于后一半。尽管是正确的, 但这个方法做了很多无用功。还记得八皇后问题中是怎么判断合法性的吗? 判断当前皇后是否和前面的皇后冲突, 但并不判断以前的皇后是否相互冲突——那些皇后在以前已经判断过了。同样的道理, 我们只需要判断当前串的后缀, 而非所有子串。

提示 7-13: 在回溯法中, 应注意避免不必要的判断, 就像在八皇后问题中那样, 只需判断新皇后和之前的皇后是否冲突, 而不必判断以前的皇后是否相互冲突。

程序如下：

```
int dfs(int cur){ //返回 0 表示已经得到解，无须继续搜索
    if(cnt++ == n){
        for(int i = 0; i < cur; i++) printf("%c", 'A'+S[i]); //输出方案
        printf("\n");
        return 0;
    }
    for(int i = 0; i < L; i++){
        S[cur] = i;
        int ok = 1;
        for(int j = 1; j*2 <= cur+1; j++){ //尝试长度为 j*2 的后缀
            int equal = 1;
            for(int k = 0; k < j; k++) //检查后半是否等于前半
                if(S[cur-k] != S[cur-k-j]) { equal = 0; break; }
            if(equal) { ok = 0; break; } //后半等于前半，方案不合法
        }
        if(ok) if(!dfs(cur+1)) return 0; //递归搜索。如果已经找到解，则直接退出
    }
    return 1;
}
```

有意思的是， $L=2$ 时一共只有 6 个串；当 $L \geq 3$ 时就很少回溯了。事实上，当 $L=3$ 时，可以构造出无限长的串，不存在相邻重复子串。

例题 7-6 带宽 (Bandwidth, UVa 140)

给出一个 n ($n \leq 8$) 个结点的图 G 和一个结点的排列，定义结点 i 的带宽 $b(i)$ 为 i 和相邻结点在排列中的最远距离，而所有 $b(i)$ 的最大值就是整个图的带宽。给定图 G ，求出让带宽最小的结点排列，如图 7-7 所示。

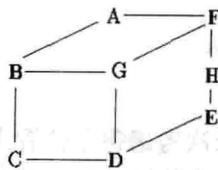


图 7-7 图 G

下面两个排列的带宽分别为 6 和 5。具体来说，图 7-8 (a) 中各个结点的带宽分别为 6, 6, 1, 4, 1, 1, 6, 6，图 7-8 (b) 中各个结点的带宽分别为 5, 3, 1, 4, 3, 5, 1, 4。

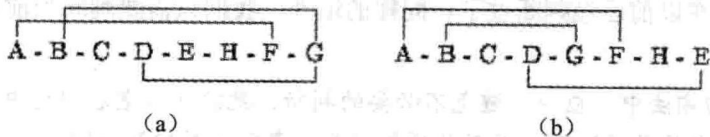


图 7-8 两个排列的带宽

【分析】

如果不考虑效率，本题可以递归枚举全排列，分别计算带宽，然后选取最小的一种方案。能否优化呢？和八皇后问题不同的是：八皇后问题有很多可行性约束（feasibility constraint），可以在得到完整解之前避免扩展那些不可行的结点，但本题并没有可行性约束——任何排列都是合法的。难道只能扩展所有结点吗？当然不是。

可以记录下目前已经找到的最小带宽 k 。如果发现已经有某两个结点的距离大于或等于 k ，再怎么扩展也不可能比当前解更优，应当强制把它“剪”掉，就像园丁在花园里为树修剪枝叶一样，也可以为解答树“剪枝（prune）”。

除此之外，还可以剪掉更多的枝叶。如果在搜索到结点 u 时， u 结点还有 m 个相邻点没有确定位置，那么对于结点 u 来说，最理想的情况就是这 m 个结点紧跟在 u 后面，这样的结点带宽为 m ，而其他任何“非理想情况”的带宽至少为 $m+1$ 。这样，如果 $m \geq k$ ，即“在最理想的情况下都不能得到比当前最优解更好的方案”，则应当剪枝。

提示 7-14：在求最优解的问题中，应尽量考虑最优性剪枝。这往往需要记录下当前最优解，并且想办法“预测”一下从当前结点出发是否可以扩展到更好的方案。具体来说，先计算一下最理想情况可以得到怎样的解，如果连理想情况都无法得到比当前最优解更好的方案，则剪枝。

例题 7-7 天平难题（Mobile Computing, ACM/ICPC Tokyo 2005, UVa1354）

给出房间的宽度 r 和 s 个挂坠的重量 w_i 。设计一个尽量宽（但宽度不能超过房间宽度 r ）的天平，挂着所有挂坠。

天平由一些长度为 1 的木棍组成。木棍的每一端要么挂一个挂坠，要么挂另外一个木棍。如图 7-9 所示，设 n 和 m 分别是两端挂的总重量，要让天平平衡，必须满足 $n \cdot a = m \cdot b$ 。

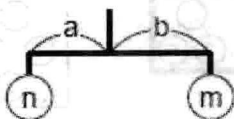
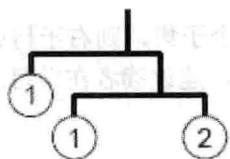
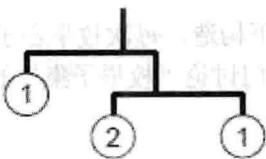


图 7-9 天平

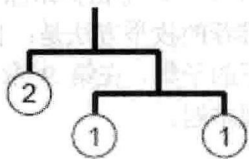
例如，如果有 3 个重量分别为 1, 1, 2 的挂坠，有 3 种平衡的天平，如图 7-10 所示。



$$\text{width} = 1 + (1/3)$$



$$\text{width} = 1 + (2/3)$$



$$\text{width} = 1 + (1/2)$$

图 7-10 3 种平衡的天平

挂坠的宽度忽略不计，且不同的子天平可以相互重叠。如图 7-11 所示，宽度为 $(1/3)+1+(1/4)$ 。

输入第一行为数据组数。每组数据前两行为房间宽度 r 和挂坠数目 s ($0 < r < 10$, $1 \leq s \leq$